

PDS-CREACIONALES: PROTOTYPE



ING. ZOILA RUIZ, PHD

PROTOTYPE

Es un patrón de diseño creacional que nos permite copiar objetos existentes sin que el código dependa de sus clases. Basa su funcionalidad en la clonación de objetos, los que son creados a partir de objetos creados previamente, simples y complejos, sin acoplarse a sus clases específicas.

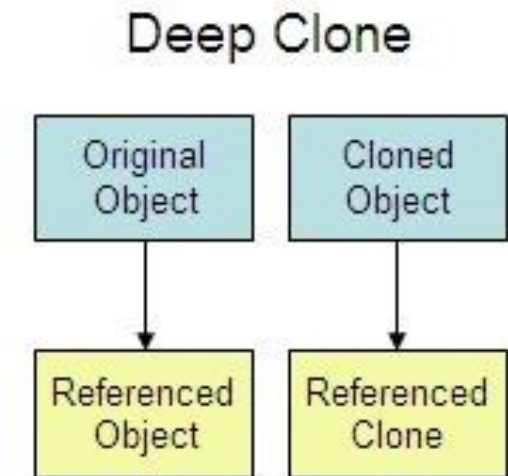
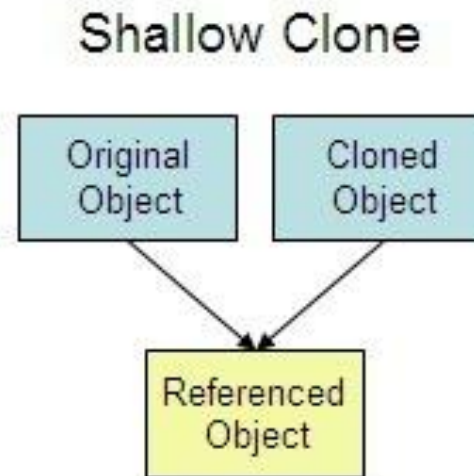
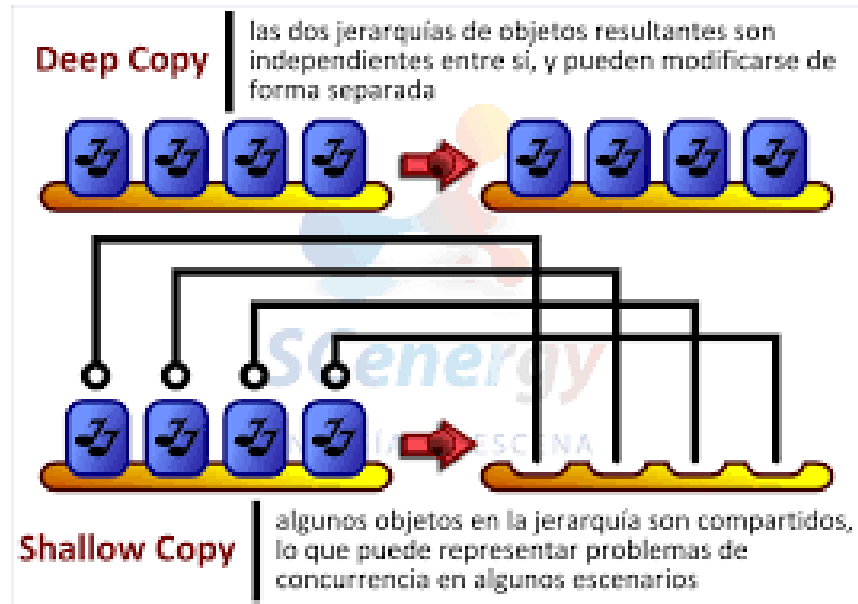
Este enfoque permite una creación de objetos más dinámica, eficiente y flexible, especialmente en escenarios donde las instancias de objetos tienen estados similares o comparten una parte importante de sus datos.

Una decisión clave al implementar el patrón prototipo es la elección entre copia profunda y superficial.

PROTOTYPE

Copia profunda: (Deep copyng) crea un nuevo objeto y copia recursivamente todos los objetos y estructuras anidados

Copia superficial: (shallow copyng) solo duplica el objeto de nivel superior y hace referencia a las estructuras anidadas originales



PROTOTYPE

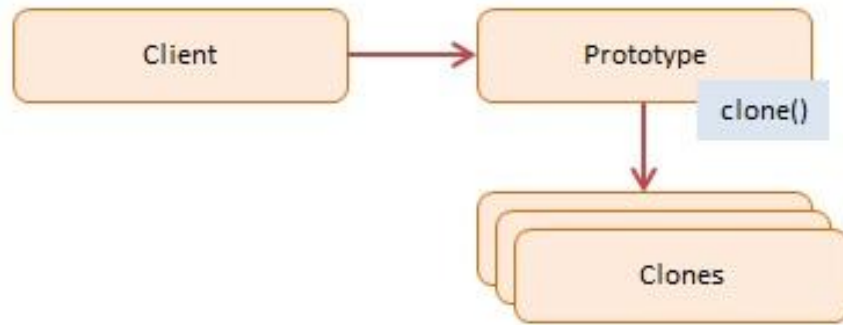
Contexto: Crear una nueva instancia puede ser costoso (por ejemplo, por tiempo o recursos).

Problema: ¿Cómo crear nuevos objetos basados en objetos existentes sin conocer sus clases concretas?

Solución: Crear nuevos objetos clonando un objeto prototipo ya existente que implementa una interfaz de clonación.

CONSECUENCIAS POSITIVAS	CONSECUENCIAS NEGATIVAS
Copia objetos rápidamente sin conocer su clase. Facilita la creación de objetos complejos	Difícil implementar correctamente el clonado profundo
Puedes evitar un código de inicialización repetido clonando prototipos prefabricados.	Riesgo de compartir referencias sin querer
Puedes clonar objetos sin acoplarlos a sus clases concretas.	Clonar objetos complejos con referencias circulares puede resultar complicado.
Alternativa a la herencia al tratar con preajustes de configuración para objetos complejos	

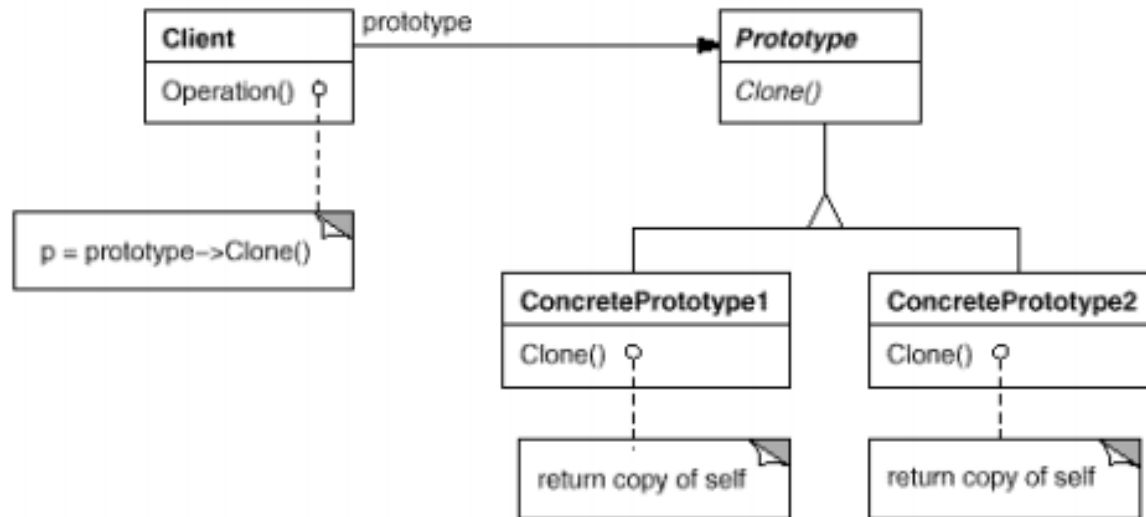
PROTOTYPE



Para la implementación del patrón se debe considerar los siguientes puntos:

- ❖ Definimos el objeto (Clase) que se desea clonar y hacemos que implemente la interfaz Cloneable .
- ❖ Se debe considerar la estructura interna de la clase para determinar que tipo de copia realizar (superficial o profunda).
- ❖ Se puede sobrescribir el método clone() para asegurar que se realice una copia profunda personalizada.

PROTOTYPE



La arquitectura del patrón tiene los siguientes elementos:

Cliente: es responsable de gestionar los prototipos y utilizar el método de clonación

IPrototype: esta interfaz define un método para clonar instancias.

ConcretePrototype: clase implementa la interfaz y proporciona la funcionalidad para crear una nueva instancia copiando una existente.

PrototypeFactory:

PROTOTYPE

Usos:

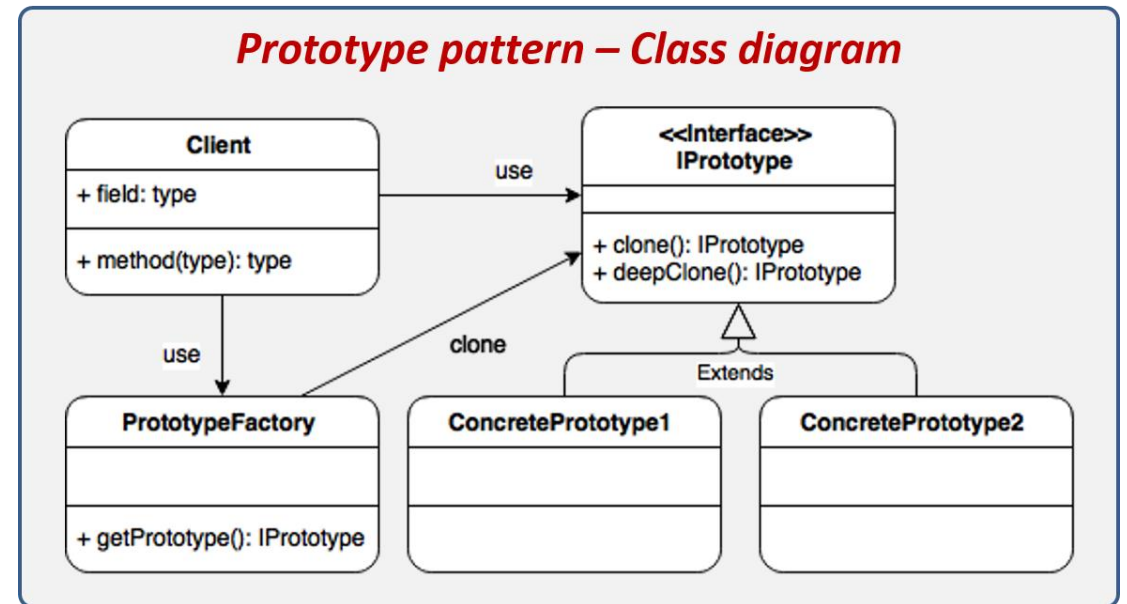
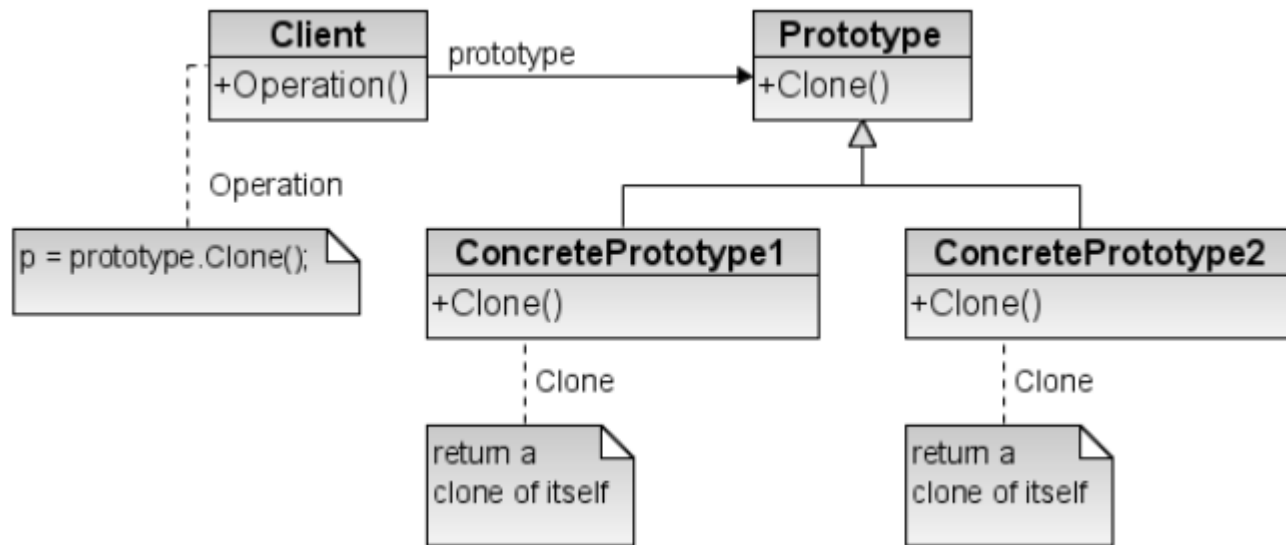
Cuando el proceso de creación de un nuevo objeto puede ser costoso o complicado, ya sea porque el objeto es complejo o porque se requieren muchos pasos para crearlo.

En lugar de crear un nuevo objeto desde cero, el patrón Prototype permite crear una copia de un objeto ya existente, modificarla según sea necesario y devolver la copia como el nuevo objeto.

Sólo existe un número pequeño de combinaciones diferentes de estado para las instancias de una clase.

Los objetos a crear tienen atributos que repiten su valor.

PROTOTYPE



PROTOTYPE
